Odds-and-Evens Game in Go

Assignment #03, Exercise 2 — Concurrent and Distributed Programming (PCD), a.y. 2025–2026

Giovanni Paradisi

November 2026

1 Analysis of the problem

A tournament of the *Odds-and-Evens* game is played by $N = 2^m$ players organised in m rounds. In each game two players show a number of fingers at the same time; if the sum is odd the player playing "odds" wins, otherwise the one playing "evens" wins. In every round the games run concurrently and the winners move on, until the final winner. The solution must use a message-passing interaction model, with no shared memory. The aspects that matter from a concurrent point of view are:

- **Natural concurrency.** The N players and the $N - 1$ games form a binary tree. Games of the same round are independent, so up to $N/2$ games can run in parallel. Moreover the two halves of the bracket are independent sub-tournaments that meet only in the final.
- **Synchronisation = rendezvous of two players.** A game can be decided only when both picks are available.
- **Data dependencies, no global barrier.** A game of round $r + 1$ depends only on the two games of round r that feed it. It can start as soon as its two players are ready, even if other games of round r are still running.
- **No shared state.** Everything a process needs travels in messages (the pick, the identity of the player and the channel where the outcome must be sent). Each game decides on its own local data.
- **Non-determinism.** Picks are random and the order in which two players reach a game is not predictable. The outcome must not depend on this order: the game is fair only if odd and even sums are equally likely.
- **Safety and liveness.** The system must be deadlock-free, every process must terminate (no leaked goroutines) and the champion must be communicated to the main program.

2 Strategy adopted

2.1 Interaction model

The solution follows the synchronous message-passing model (CSP), directly supported by Go: goroutines are the *processes* and unbuffered channels give synchronous *send/receive*. The interaction is the **client-server** pattern: players are clients, each match is an **active server** (resource-manager style) with a many-to-one *request* channel and a reply channel per client.

2.2 Processes and channels

There are N **Player** processes, $N - 1$ **Match** processes (the last one is the **Final**, which also announces the champion) and the **main** process that creates the channels, starts the processes and waits for the champion.

Entity	Role	Communication	Lifetime
Player (N)	client	sends `request` (pick, id, reply channel) to the match of the current round; receives the outcome on its own `reply` channel	until it loses or wins the final
Match ($N-1$), implemented by `referee`	active server (referee)	receives exactly two requests on its own request channel (many-to-one); sends the outcome to both players through the reply channels carried in the messages	one game, then terminates
Final (1)	last Match	as a Match; in addition sends the winner's id on the `champion` channel	one game
main (1)	coordinator	creates the channels, gives each player its own path, starts the processes, receives the champion	until the champion is announced

Table 1: Entities of the system and their interaction.

2.3 Tournament topology

The bracket is static. Player i plays, in round r, the match number $\lfloor i/2^r \rfloor$; the *path* of a player is the list of the request channels of the matches it would play if it keeps winning. **main** creates all the channels and gives each player only its own path, so no data structure is shared among processes. The two halves of the bracket correspond to the player ranges $[0, N/2)$ and $[N/2, N)$, which share no channel until the final. Figure 1 shows the structure for $N = 8$, Figure 2 the interaction of one game and Figure 3 the life cycle of a player.

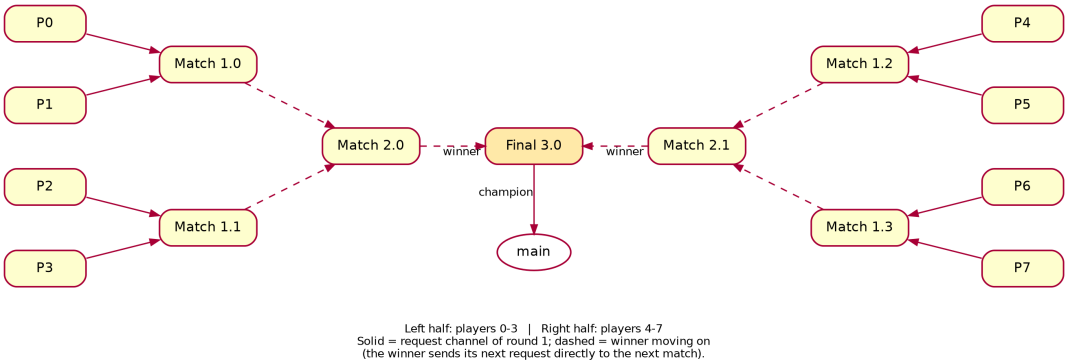


Figure 1: Structure of the tournament for $N = 8$.

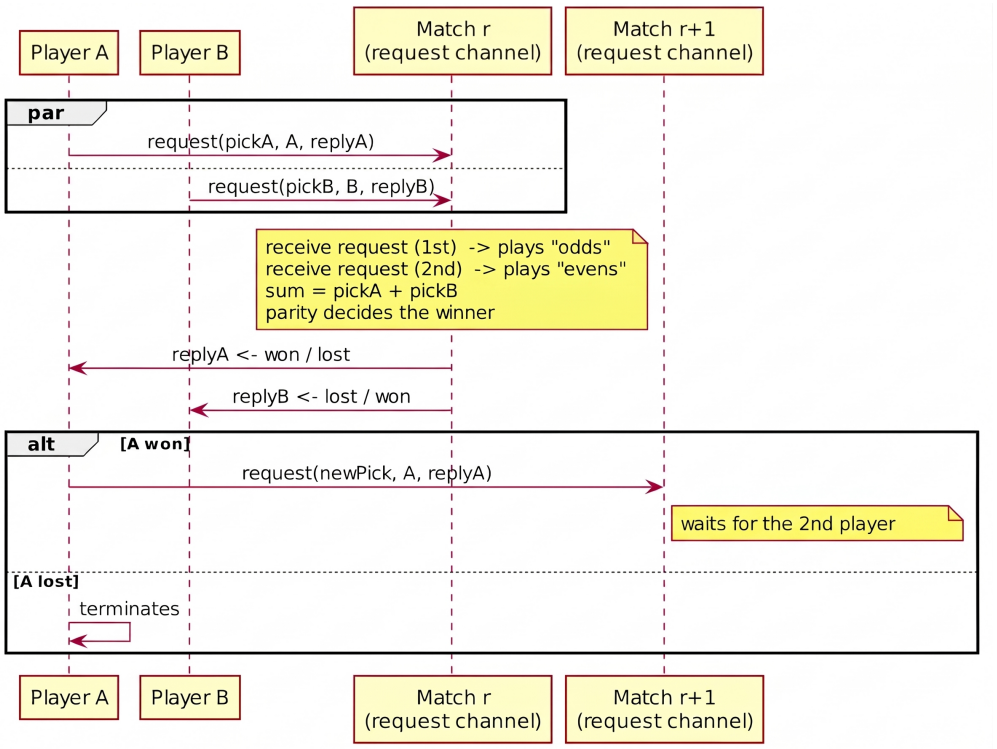


Figure 2: Sequence diagram of one game and of the move of the winner to the next round.

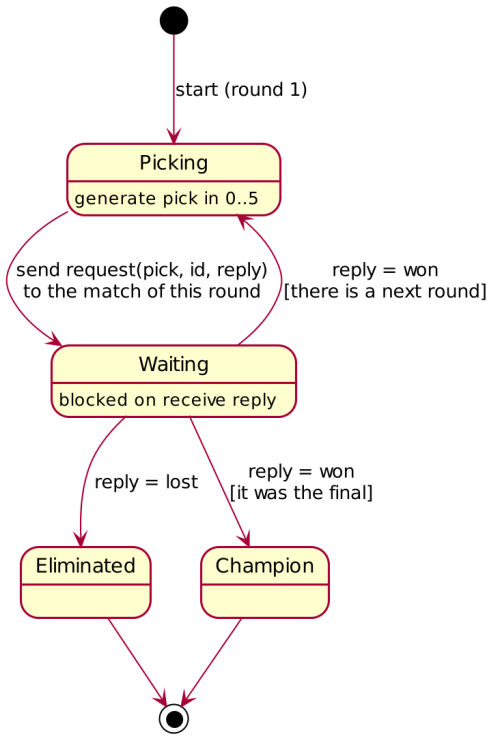


Figure 3: State diagram of a Player process.

2.4 Why it is correct

- **No shared memory.** Processes interact only through *send* and *receive*; the reply channel travels inside the request message.
- **Fairness.** Picks are uniform in 0..5, so $P(sumodd) = P(sumeven) = 1/2$ whatever the role as-

signment; giving the "odds" role to the first request that arrives is therefore harmless and keeps the non-determinism of the many-to-one channel.

- **Deadlock freedom.** Each request channel has exactly two senders and one receiver that performs exactly two receives, so no sender blocks forever. A match replies to two players that are both waiting on their reply channel, and a game of round $r+1$ waits only for winners of round r, so there is no cyclic wait.
- **Termination.** A match ends after one game, a loser after its reply, the champion after the final, and `main` after receiving the champion: all $2N-1$ processes terminate.

3 Implementation and validation

The solution is written in Go and is organised in three files: `player.go` (the `Player` client and the message type), `match.go` (the `Match`/`Final` server logic) and `main.go` (creation of the channels, start-up of the processes and collection of the champion). Goroutines implement the processes and unbuffered channels implement the synchronous communication. The excerpt below, simplified by removing logging and input checks, shows the core of the implementation.

```
type Msg struct {                       // request sent by a player to a match
    value    int                        // fingers shown: 0..5
    playerID string
    reply    chan bool                  // the match answers here (true = won)
}

func Player(id int, path []chan<- Msg) { // path: one request channel per round
    reply := make(chan bool)
    for _, request := range path {
        request <- Msg{rand.IntN(6), name(id), reply}
        if !<-reply { return }          // eliminated
    }
}

func referee(request <-chan Msg) Msg {   // body of a match: serves two requests
    odd  := <-request                    // first arrived plays "odds"
    even := <-request                    // second arrived plays "evens"
    winner, loser := even, odd
    if (odd.value+even.value)%2 == 1 { winner, loser = odd, even }
    winner.reply <- true
    loser.reply  <- false
    return winner
}

// main: requests[r][j] is the request channel of match j of round r+1
for r := 0; r < m; r++ { /* create N/2^(r+1) channels */ }
for each match (r, j) { go Match(requests[r][j]) }      // the last one is Final
for i := 0; i < N; i++ {
    for r := range path { path[r] = requests[r][i>>(r+1)] }
    go Player(i, path)
}
fmt.Println("Tournament winner:", <-champion)
```

Run with `go run . -m 4` (16 players, m between 1 and 10). The code was checked with the Go race detector for m up to 10 and with repeated runs without failures. Example of output for $m = 4$:

```
[Round 1 | Match 1] player-2(0, odds) vs player-3(0, evens) -> sum=0 -> player-3 wins, player-2 out
[Round 1 | Match 2] player-4(3, odds) vs player-5(5, evens) -> sum=8 -> player-5 wins, player-4 out
[Round 1 | Match 3] player-6(4, odds) vs player-7(4, evens) -> sum=8 -> player-7 wins, player-6 out
[Round 1 | Match 4] player-8(2, odds) vs player-9(4, evens) -> sum=6 -> player-9 wins, player-8 out
[Round 1 | Match 5] player-10(0, odds) vs player-11(1, evens) -> sum=1 -> player-10 wins, player-11 out
[Round 1 | Match 6] player-12(1, odds) vs player-13(0, evens) -> sum=1 -> player-12 wins, player-13 out
[Round 1 | Match 7] player-15(3, odds) vs player-14(2, evens) -> sum=5 -> player-15 wins, player-14 out
[Round 1 | Match 0] player-0(5, odds) vs player-1(0, evens) -> sum=5 -> player-0 wins, player-1 out
[Round 2 | Match 2] player-9(1, odds) vs player-10(0, evens) -> sum=1 -> player-9 wins, player-10 out
[Round 2 | Match 1] player-5(2, odds) vs player-7(4, evens) -> sum=6 -> player-7 wins, player-5 out
[Round 2 | Match 3] player-12(5, odds) vs player-15(3, evens) -> sum=8 -> player-15 wins, player-12 out
[Round 3 | Match 1] player-9(1, odds) vs player-15(0, evens) -> sum=1 -> player-9 wins, player-15 out
[Round 2 | Match 0] player-3(0, odds) vs player-0(3, evens) -> sum=3 -> player-3 wins, player-0 out
[Round 3 | Match 0] player-3(0, odds) vs player-7(2, evens) -> sum=2 -> player-7 wins, player-3 out
[Round 4 | Match 0] player-9(5, odds) vs player-7(0, evens) -> sum=5 -> player-9 wins, player-7 out
Tournament winner: player-9
```

Each game logs its line before replying, so the lines of a round always appear after the two games that feed it, while games of the same round may appear in any order, which shows their concurrency.